

NAME: Shaikh Areeba Mohammed Ismail

CLASS/BATCH: TE-B2

ROLL NO: 46

SUBJECT: Artificial Intelligence

GROUP: A

ASSIGNMENT NO: 02

Problem Statement: Implement A star Algorithm for any game search problem

A* Version - 1

```
# we use a tree data structure to solve a star problem
# Each node in the tree is an object of class Node
# Each object of class node is a form of the matrix
class Node:
    def __init__(self, data, level, fval):
        # Initialize the node with the data, level of the node
        # and the calculated fvalue
        self.data = data
        self.level = level
        self.fval = fval

    def generate_child(self):
        """ Generate child nodes from the given node by moving
        the blank space
            either in the four directions {up,down,left,right}
        """
        x, y = self.find(self.data, '_')
        """ val_list contains position values for moving the
        blank space in either of
            the 4 directions [up,down,left,right] respectively.
        """

        val_list = [[x, y-1], [x, y+1], [x-1, y], [x+1, y]]
        #this are the x and y coordinates of the children in the matrix
        children = []
        #val list contain 4 values we check the values in the
        start game matrix
        for i in val_list:
            child = self.shuffle(self.data, x, y, i[0], i[1])
            if child is not None: #we assign a child node if and
            only if the child exists in the matrix
```

```

        child_node = Node(child, self.level+1, 0)
        children.append(child_node)
    return children

    def shuffle(self, puz, x1, y1, x2, y2):

        # Move the blank space in the given direction and if the
        position value are out of limits the return None
        if x2 >= 0 and x2 < len(self.data) and y2 >= 0 and y2 <
len(self.data):
            temp_puz = []
            temp_puz = self.copy(puz)
            temp = temp_puz[x2][y2]
            temp_puz[x2][y2] = temp_puz[x1][y1] #moving the
blank space (exchange values in matrix)
            temp_puz[x1][y1] = temp
            return temp_puz
        else:
            return None

    def copy(self, root):
        # Copy function to create a similar matrix of the given
node
        temp = []
        for i in root:
            t = []
            for j in i:
                t.append(j)
            temp.append(t)
        return temp

    def find(self, puz, x):
        # Specifically used to find the position of the blank
space
        for i in range(0, len(self.data)):
            for j in range(0, len(self.data)):
                if puz[i][j] == x:
                    return i, j

class Puzzle:
    def __init__(self, size):

        # Initialize the puzzle size by the specified size, open
and closed lists to empty
        self.n = size
        self.open = []

```

```

        self.closed = []

def accept(self):

    # Accepts the puzzle from the user
    puz = []
    for i in range(0, self.n):
        temp = input().split(" ")
        puz.append(temp)
    return puz

def f(self, start, goal):
    # Heuristic Function to calculate heuristic value f(x)
    = h(x) + g(x)
    return self.h(start.data, goal)+start.level

def h(self, start, goal):
    # Calculates the different between the given puzzles
    temp = 0
    for i in range(0, self.n):
        for j in range(0, self.n):
            if start[i][j] != goal[i][j] and start[i][j] !=
'_' :
                temp += 1
    return temp

def process(self):
    # Accept Start and Goal Puzzle state
    print("Enter the start state matrix \n")
    start = self.accept()
    print("Enter the goal state matrix \n")
    goal = self.accept()

    # we initiate the tree with the start configuration of
the matrix
    start = Node(start, 0, 0)
    start.fval = self.f(start, goal)
    # Put the start node in the open list
    self.open.append(start)
    print("\n\n")
    while True:
        cur = self.open[0]
        print("")
        print("  | ")
        print("  | ")
        print(" \\\\'/")

```

```

        print("  V  \n")
        for i in cur.data:
            for j in i:
                print(j, end=" ")
            print("")
            # If the difference between current and goal node
            # is 0 we have reached the goal node
            if(self.h(cur.data, goal) == 0):
                break
        for i in cur.generate_child():
            i.fval = self.f(i, goal)
            self.open.append(i)
        self.closed.append(cur)
        del self.open[0]

        # sort the opne list based on f value ""
        self.open.sort(key=lambda x: x.fval, reverse=False)

puz = Puzzle(3)
puz.process()

```

OUTPUT:

Enter the start state matrix

```
1 2 3
5 6 _
7 8 4
```

Enter the goal state matrix

```
1 2 3
5 8 6
_ 7 4
```

```
  |
  |
 \'|/
  V
```

```
1 2 3
5 6 _
7 8 4
```

```
  |
  |
 \'|/
  V
```

```
1 2 3
5 _ 6
7 8 4
```

```
  |
  |
 \'|/
  V
```

```
1 2 3
5 8 6
7 _ 4
```

```
  |
  |
 \'|/
  V
```

```
1 2 3
5 8 6
_ 7 4
```

A* Version - 2

```
import heapq
from termcolor import colored

# Class to represent the state of the 8-puzzle
class PuzzleState:
    def __init__(self, board, parent, move, depth, cost):
        self.board = board # The puzzle board configuration
        self.parent = parent # Parent state
        self.move = move # Move to reach this state
        self.depth = depth # Depth in the search tree
        self.cost = cost # Cost (depth + heuristic)

    def __lt__(self, other):
        return self.cost < other.cost

# Function to display the board in a visually appealing format
def print_board(board):
    print("+---+---+---+")
    for row in range(0, 9, 3):
        row_visual = "|"
        for tile in board[row:row + 3]:
            if tile == 0: # Blank tile
                row_visual += f" {colored(' ', 'cyan')} |"
            else:
                row_visual += f" {colored(str(tile),
'yellow')} |"
        print(row_visual)
        print("+---+---+---+")

# Goal state for the puzzle
goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]

# Possible moves for the blank tile (up, down, left, right)
moves = {
    'U': -3, # Move up
    'D': 3, # Move down
    'L': -1, # Move left
    'R': 1 # Move right
}

# Function to calculate the heuristic (Manhattan distance)
def heuristic(board):
    distance = 0
    for i in range(9):
        if board[i] != 0:
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(board[i] - 1, 3)
```

```

        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

# Function to get the new state after a move
def move_tile(board, move, blank_pos):
    new_board = board[:]
    new_blank_pos = blank_pos + moves[move]
    new_board[blank_pos], new_board[new_blank_pos] =
new_board[new_blank_pos], new_board[blank_pos]
    return new_board

# A* search algorithm
def a_star(start_state):
    open_list = []
    closed_list = set()
    heapq.heappush(open_list, PuzzleState(start_state, None,
None, 0, heuristic(start_state)))

    while open_list:
        current_state = heapq.heappop(open_list)

        if current_state.board == goal_state:
            return current_state

        closed_list.add(tuple(current_state.board))

        blank_pos = current_state.board.index(0)

        for move in moves:
            if move == 'U' and blank_pos < 3: # Invalid move
up
                continue
            if move == 'D' and blank_pos > 5: # Invalid move
down
                continue
            if move == 'L' and blank_pos % 3 == 0: # Invalid
move left
                continue
            if move == 'R' and blank_pos % 3 == 2: # Invalid
move right
                continue

            new_board = move_tile(current_state.board, move,
blank_pos)

            if tuple(new_board) in closed_list:
                continue

```

```

        new_state = PuzzleState(new_board, current_state,
move, current_state.depth + 1, current_state.depth + 1 +
heuristic(new_board))
        heapq.heappush(open_list, new_state)

    return None

# Function to print the solution path
def print_solution(solution):
    path = []
    current = solution
    while current:
        path.append(current)
        current = current.parent
    path.reverse()

    for step in path:
        print(f"Move: {step.move}")
        print_board(step.board)

# Initial state of the puzzle
initial_state = [1, 2, 3, 4, 0, 5, 6, 7, 8]

# Solve the puzzle using A* algorithm
solution = a_star(initial_state)

# Print the solution
if solution:
    print(colored("Solution found:", "green"))
    print_solution(solution)
else:
    print(colored("No solution exists.", "red"))

```

OUTPUT:

Solution found:

Move: None

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 |   | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 |   |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+
```

Move: D

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
| 6 | 7 |   |
+---+---+---+
```

Move: L

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
| 6 |   | 7 |
+---+---+---+
```

Move: L

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 8 |
+---+---+---+
|   | 6 | 7 |
+---+---+---+
```

Move: U

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
|   | 5 | 8 |
+---+---+---+
| 4 | 6 | 7 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 5 |   | 8 |
+---+---+---+
| 4 | 6 | 7 |
+---+---+---+
```

Move: D

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 5 | 6 | 8 |
+---+---+---+
| 4 |   | 7 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 5 | 6 | 8 |
+---+---+---+
| 4 | 7 |   |
+---+---+---+
```

Move: U

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 5 | 6 |   |
+---+---+---+
| 4 | 7 | 8 |
+---+---+---+
```

Move: L

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 5 |   | 6 |
+---+---+---+
| 4 | 7 | 8 |
+---+---+---+
```

Move: L

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
|   | 5 | 6 |
+---+---+---+
| 4 | 7 | 8 |
+---+---+---+
```

Move: D

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
|   | 7 | 8 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
| 7 |   | 8 |
+---+---+---+
```

Move: R

```
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
| 7 | 8 |   |
+---+---+---+
```